

คู่มือการเขียนโปรแกรม

โปรแกรมจำลองการตั้งค่าอุปกรณ์เราเตอร์

Router Configuring Simulator

จัดทำโดย

นางสาวณัฏฐา ลั่นทมทอง

นางสาวพนิดา ตั้งวิทย์โมไนย

อาจารย์ที่ปรึกษา

อาจารย์ ธนา หงษ์สุวรรณ

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2544

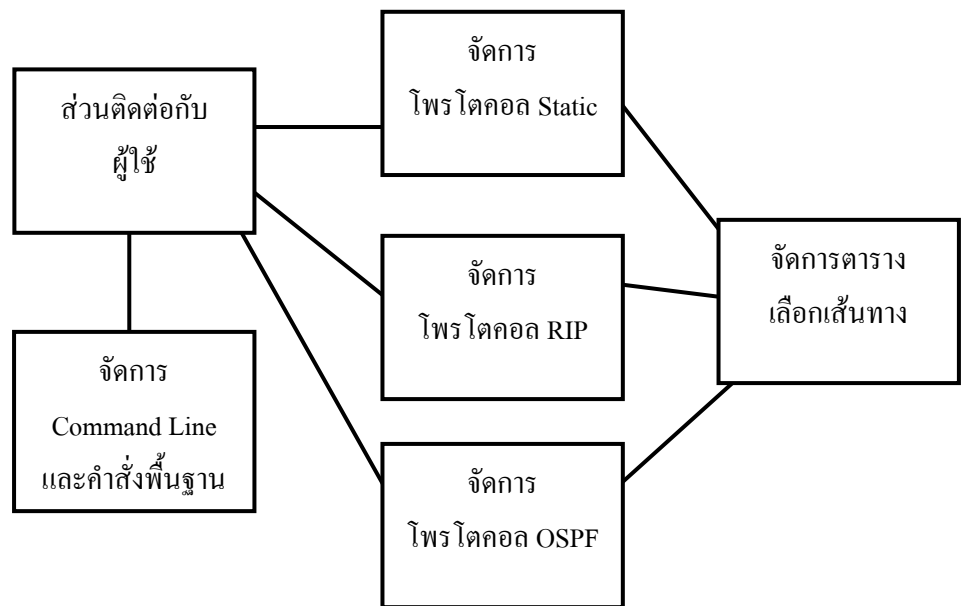
คู่มือการเขียนโปรแกรม

1. โครงสร้างของโปรแกรม (Design)

โปรแกรมนี้ออกแบบโดยใช้วิธี ออบเจกต์โอเรียนเต้ดโปรแกรมมิง (Object Oriented Programming) เพื่อให้การเขียนโปรแกรมเป็นไปได้ง่ายต่อการเพิ่มและเปลี่ยนแปลงโปรแกรม โดยแบ่งการทำงานของโปรแกรมออกเป็นส่วนย่อยๆ เป็นคลาสต่างๆ แต่ละคลาสจะมีหน้าที่การทำงานที่แตกต่างกันดังนี้

- **Frame1** เป็นคลาสสำหรับสร้างหน้าต่างอินเทอร์เฟซเพื่อติดต่อกับผู้ใช้ โดยหน้าต่างอินเทอร์เฟซจะประกอบไปด้วยเมนูบาร์ ทูลบาร์ และส่วนที่เป็นหน้าต่างทำงานของแต่ละเราเตอร์
- **Frame2** เป็นคลาสที่แสดงผลภาพรวมของระบบ รวมทั้งจัดการกับการเลื่อนตำแหน่งของภาพ
- **RouterConsole** เป็นคลาสที่เก็บหน้าต่างสำหรับทำงานของเราเตอร์แต่ละตัว
- **Router** เป็นคลาสที่ประกอบไปด้วยข้อมูลต่างๆ ของเราเตอร์เช่น ชื่อเราเตอร์ สถานะการทำงานของเราเตอร์ โหมดการทำงานของเราเตอร์ อินเทอร์เฟซของเราเตอร์ และตารางการค้นหาเส้นทางของเราเตอร์ โดยฟังก์ชันการทำงานของคลาสนี้จะมีฟังก์ชันสำหรับส่งตารางค้นหาเส้นทาง และรับตารางค้นหาเส้นทางจากเราเตอร์ตัวอื่น
- **RouterImg** เป็นคลาสที่เก็บข้อมูลเกี่ยวกับภาพของเราเตอร์แต่ละตัว
- **Interface** เป็นคลาสที่ประกอบไปด้วยข้อมูลของอินเทอร์เฟซแต่ละตัว เช่น ชื่ออินเทอร์เฟซ ไอพีแอดเดรส ประเภทของอินเทอร์เฟซ ชับเน็ต และสถานะของอินเทอร์เฟซ
- **RoutingTable** เป็นคลาสที่เก็บข้อมูลของตารางค้นหาเส้นทางเช่น เครือข่ายปลายทาง เกตเวย์ที่ออก ค่าเมตริก หรือประเภทของข้อมูลค้นหาเส้นทางว่าเป็นแบบสแตติก อาร์ไอพีหรือแบบอื่น ฟังก์ชันการทำงานหลักก็จะเป็นการเพิ่มข้อมูลลงในตารางค้นหาเส้นทาง
- **SwitchCMD** มีหน้าที่รับคำสั่งจากผู้ใช้งานมาตรวจสอบว่าเป็นคำสั่งที่ถูกต้องก่อนจะไปเลือกการทำงานของแต่ละคำสั่ง ซึ่งจะมีฟังก์ชันหลักเป็นตัวตรวจสอบว่าเป็นคำสั่งที่ถูกต้อง และส่งไปทำงานตามแต่ละคำสั่ง แต่จะมีเป็นฟังก์ชันย่อยที่ช่วยฟังก์ชันใหญ่ในการตรวจสอบ เช่น ฟังก์ชันสำหรับหาช่องว่าง
- **Command** เป็นคลาสที่จัดการเกี่ยวกับคำสั่งคำสั่งของเราเตอร์
- **Rip** เป็นคลาสที่จัดการเกี่ยวกับ การเลือกเส้นทางแบบ RIP โดยคลาสนี้มีการทำงานเป็น Threads รับส่งตารางเลือกเส้นทางเป็นช่วงเวลาที่ตั้งไว้ตามข้อกำหนดของโปรโตคอล
- **Ospf** เป็นคลาสที่จัดการเกี่ยวกับ การเลือกเส้นทางแบบ OSPF มีการทำงานเป็น Threads

โดยคลาสต่างๆ นี้สามารถแบ่งตามการจัดการได้ดังนี้

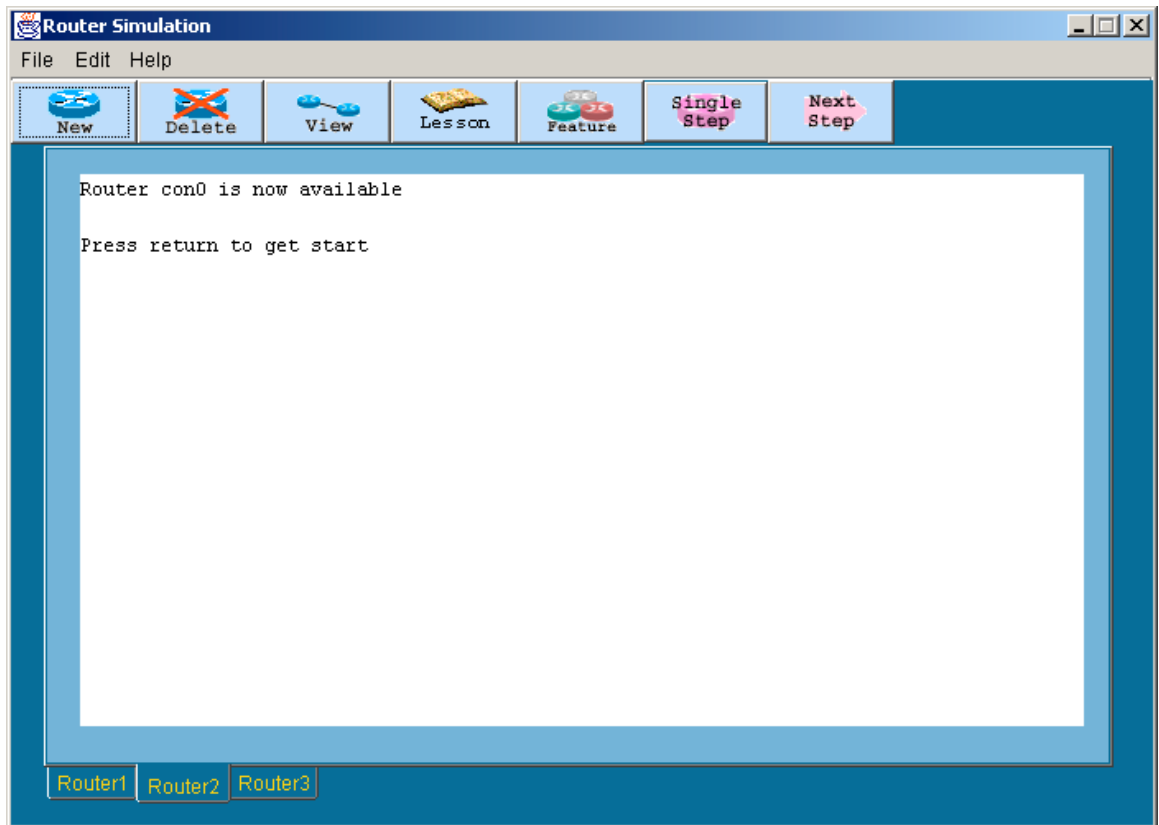


รูปที่ 1การออกแบบโปรแกรม

2. การจัดการส่วนติดต่อผู้ใช้ (Graphic User Interface : GUI)

ส่วนของ GUI จะมี 3 ส่วนหลักๆ คือ

1. ส่วนหน้าจอหลักของโปรแกรม ประกอบด้วยทูลบาร์และส่วนของแท็บเพน หน้าจอแท็บเพนแต่ละแท็บจะแทนหน้าจอคอนโซลของเราเตอร์แต่ละตัว การออกแบบส่วนนี้จะแบ่งเป็น 2 คลาส คือ คลาส Frame1 ซึ่งเป็นหน้าจอหลัก และคลาส RouterConsole ซึ่งเป็นส่วนของหน้าจอคอนโซลของเราเตอร์ 1 ตัว เมื่อมีการเพิ่มเราเตอร์เข้าไปในระบบก็เป็นการสร้างออบเจกต์ของ RouterConsole เพิ่มขึ้น



รูปที่ 2 หน้าจอหลักของโปรแกรม

ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส Frame1 และ RouterConsole มีฟังก์ชันการทำงานดังนี้

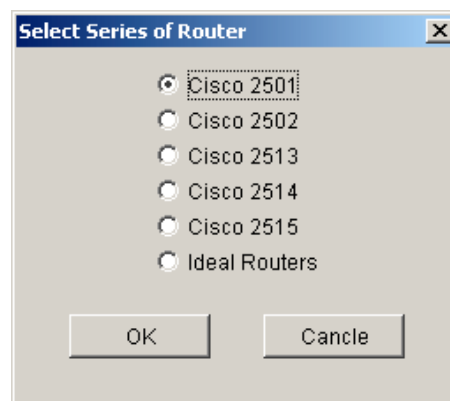
คลาส Frame1

- Frame1() เป็นคอนสตรัคเตอร์ของคลาส ในฟังก์ชันนี้จะแสดงหน้าจอโปรแกรมหลักทั้งส่วนของเมนูบาร์ ทูลบาร์ และแท็บเพน
- AddTab() เป็นฟังก์ชันสำหรับเพิ่มหน้าแท็บเพน เมื่อผู้ใช้กดปุ่มเพิ่มเราเตอร์
- SetTab() เป็นฟังก์ชันสำหรับกำหนดข้อความที่แสดงตรงแท็บเพน จะใช้ในกรณีที่ผู้ใช้ใช้คำสั่งเปลี่ยนชื่อของเราเตอร์
- ส่วนฟังก์ชันที่เหลือเป็นฟังก์ชันสำหรับการกระทำของผู้ใช้สำหรับแต่ละคอมโพเนนต์ ซึ่งประเภทของการจัดการกับการกระทำของผู้ใช้ออกเป็น 2 ประเภท คือการกระทำที่ผู้ใช้กระทำกับคอมโพเนนต์หนึ่งๆ และการกระทำที่เกี่ยวกับคีย์บอร์ดในขณะที่คอมโพเนนต์หนึ่งๆ ถูกโฟกัสอยู่ เช่น jButton_actionPerformed() เป็นการจัดการกับการกระทำของผู้ใช้ที่กับปุ่มเพิ่มเราเตอร์

และ `jNewButton_keyPressed()` จัดการการกระทำของการกดคีย์บอร์ดขณะที่ปุ่มเพิ่มเราเตอร์ถูก
โฟกัสอยู่

คลาส `RouterConsole`

- `RouterConsole()` เป็นคอนสตรัคเตอร์ของคลาสทำหน้าที่แสดงส่วนที่จะนำหน้าจอไปวางบนแท็บ
เพน
 - `EnterKey()` เป็นฟังก์ชันที่จัดการกับการทำงานของคีย์ `Enter` เพื่อเริ่มต้นการทำงาน เนื่องจากเรา
เตอร์สามารถกำหนดระดับความปลอดภัยของผู้ใช้ได้ เมื่อผู้สมัครคีย์ `Enter` ก็สามารถทำงานกับเรา
เตอร์ได้เลยหรือจะต้องใส่พาสเวิร์ดก่อนถึงจะเริ่มทำงานได้
 - `ShowPrompt()` เป็นฟังก์ชันสำหรับแสดงพรอมต์ โดยจะดูจากโหมดการทำงานของเราเตอร์แล้ว
จึงแสดงพรอมต์ของโหมดการทำงานนั้นๆ สำหรับฟังก์ชันนี้จะรับอาร์กิวเมนต์เข้าไปเป็นสตริง
เพื่อนำไปแสดงผลต่อจากพรอมต์
 - ส่วนฟังก์ชันที่เหลือเป็นฟังก์ชันที่จัดการกับการกระทำที่เกี่ยวกับคีย์บอร์ดและเมาส์ เช่น ฟังก์ชัน
`jTextArea1_keyPressed()` จัดการการกระทำที่เกี่ยวกับคีย์บอร์ดในขณะที่ `TextArea` ถูกโฟกัส
ฟังก์ชัน `jTextArea1_mouseClicked()` จัดการการกระทำของการคลิกเมาส์ในขณะที่ `TextArea` ถูก
โฟกัส และฟังก์ชัน `jTextArea1_mouseReleased()` จัดการกับการกระทำของการปล่อยเมาส์ ทำให้
ผู้ใช้ไม่สามารถเลือกข้อความได้
2. หน้าจอสำหรับเลือกรุ่นของเราเตอร์ เมื่อเพิ่มเราเตอร์เข้าไปในระบบ ผู้ใช้จะต้องเลือกรุ่นของเราเตอร์
ที่ต้องการ ซึ่งแต่ละรุ่นก็จะมีอินเทอร์เน็ตเฟสที่ต่างกัน

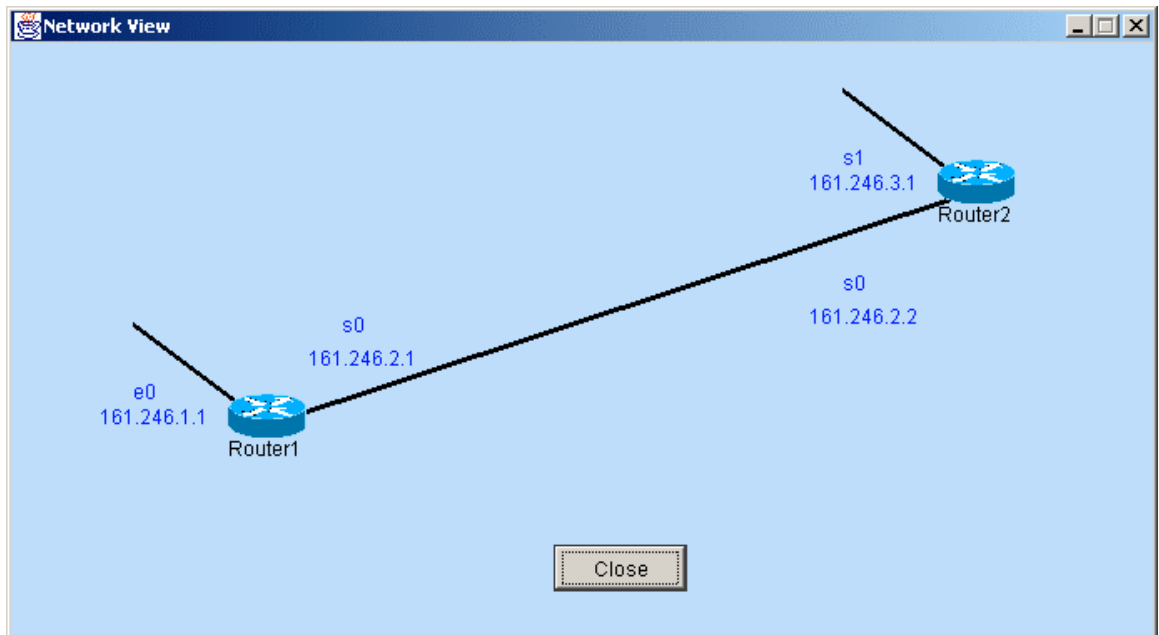


รูปที่ 3 หน้าจอสำหรับเลือกรุ่นของเราเตอร์

ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส `Dialog1` มีฟังก์ชันการทำงานดังนี้

- `Dialog1()` เป็นคอนสตรัคเตอร์สำหรับแสดงหน้าจอไดอะล็อกนี้ ประกอบไปด้วย `RadioButton` ที่
จัดเป็นกลุ่มเดียวกันเพื่อให้ผู้ใช้เลือกตัวเลือกได้เพียงตัวเลือกเดียวเท่านั้น และปุ่ม `OK` กับ `Cancel`
สำหรับตกลงหรือยกเลิกการเลือกตัวเลือก

- `getSelectChoice()` เป็นฟังก์ชันสำหรับตรวจสอบว่าผู้ใช้เลือกตัวเลือกใด แล้วให้ค่าของตัวเลือกที่ผู้ใช้เลือกออกมาเป็นค่าตัวเลข
 - `jButton1_actionPerformed()` เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้กับปุ่ม OK
 - `jButton2_actionPerformed()` เป็นฟังก์ชันสำหรับจัดการการกระทำของผู้ใช้กับปุ่ม Cancel
3. หน้าจอสำหรับแสดงภาพรวมของเครือข่าย ใช้ `Frame` ในการแสดงผล จะแสดงการเชื่อมต่อระหว่างเราเตอร์ตามที่กำหนดให้กับเราเตอร์แต่ละตัว



รูปที่ 4 ภาพแสดงภาพรวมของเครือข่าย

ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส `Frame2` และ `Panel2` มีฟังก์ชันการทำงานดังนี้

คลาส `Frame2`

- `Frame2()` เป็นคอนสตรัคเตอร์ของคลาสสำหรับแสดงผลภาพรวมของเครือข่าย

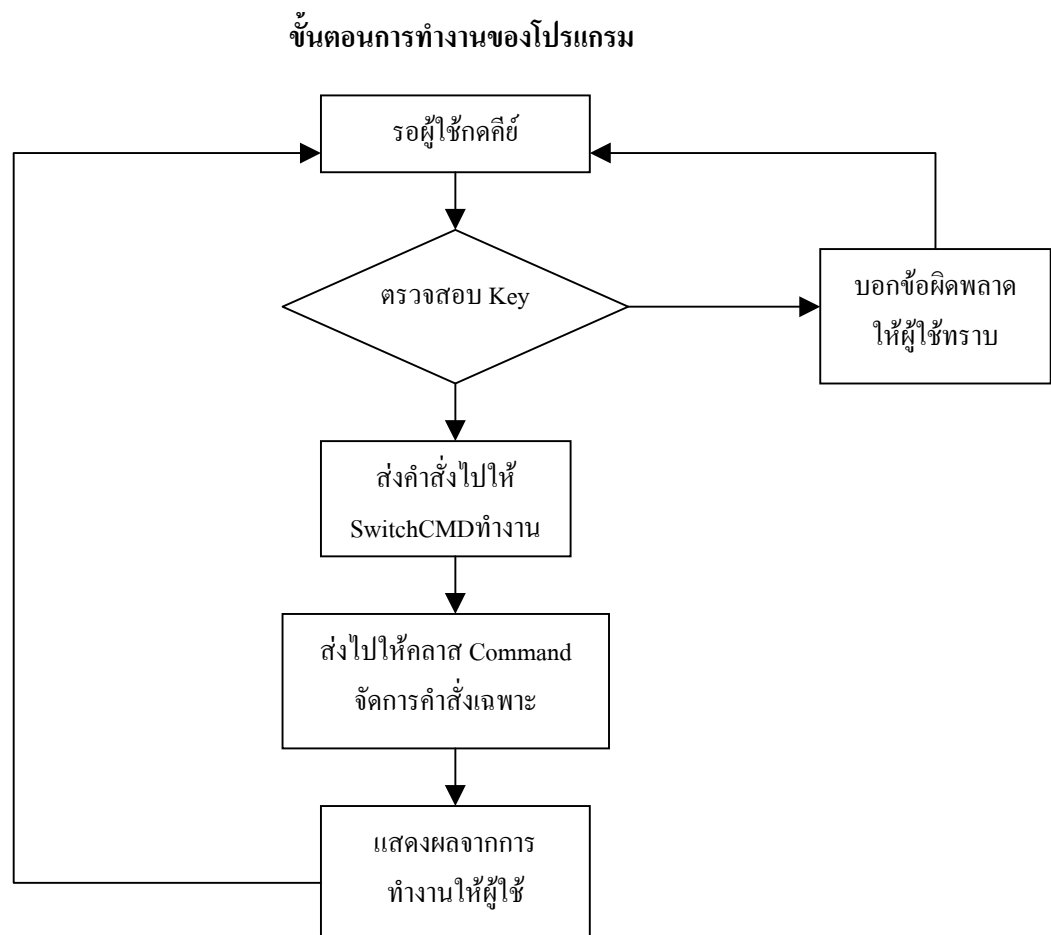
คลาส `Panel2`

- `Panel2()` เป็นคอนสตรัคเตอร์ของคลาส ฟังก์ชันนี้จะแสดงเฉพาะส่วนพื้นของหน้าจอ
- `paintComponent()` เป็นฟังก์ชันสำหรับวาดภาพเราเตอร์ และลากเส้นของแต่ละอินเทอร์เฟซ
- `this_mousePressed()` เป็นฟังก์ชันจัดการกับการคลิกเมาส์
- `this_mouseReleased()` เป็นฟังก์ชันจัดการกับการปล่อยเมาส์
- `this_mouseDragged()` เป็นฟังก์ชันจัดการกับการคลิกเมาส์และเลื่อนเมาส์ไปพร้อมกัน

ความสัมพันธ์ของหน้าจอทั้ง 3 ส่วนนี้คือ มีหน้าจอหลักเป็นหลักของโปรแกรมรอรับคำสั่งจากผู้ใช้ แล้วทำงานตามที่ผู้ใช้ระบุโดยส่งคำสั่งที่รับเข้ามานี้ให้ส่วนจัดการ `Command Line` ทำงานต่อ

3. การจัดการส่วนของ Command line

มีคลาสที่จัดการส่วนนี้ 1 คลาสคือ คลาส SwitchCMD จะคอยรับคำสั่งที่ส่งมาจากคลาส Frame1 ที่จัดการในส่วนที่ติดต่อกับผู้ใช้ เมื่อรับคำสั่งมาแล้วจะนำมาตรวจสอบว่าเป็นคำสั่งที่ถูกต้อง สามารถทำงานได้หรือไม่ ถ้าถูกต้องจะส่งคำสั่งไปทำงาน และรอรับผลการทำงาน เพื่อนำมาแสดงผลให้ผู้ใช้ทราบ



รูปที่ 5 แสดงขั้นตอนการจัดการกับส่วน *Commands line*

ฟังก์ชันการทำงานสำหรับส่วนนี้จะอยู่ในคลาส SwitchCMD และ Command มีฟังก์ชันการทำงานดังนี้

คลาส SwitchCMD

- sendCMD() เป็นฟังก์ชันหลักในการทำงานของคลาสนี้ ค่าที่รับเข้ามาในฟังก์ชันนี้จะมีอยู่ 2 อย่างคือ เราเตอร์และคำสั่งที่ต้องการทำงานกับเราเตอร์นั้นๆ โดยเริ่มต้นที่ดูไหมคการทำงานของเราเตอร์เพื่อเลือกชุดคำสั่งที่สามารถทำงานได้กับไหมคนั้น หลังจากนั้นจะตัดคำมาทีละคำเพื่อเปรียบเทียบกับคำสั่งที่มีทุกคำสั่ง ถ้าเจอคำสั่งที่สอดคล้องกันก็จะนับจำนวนคำสั่งนั้นไว้ เมื่อเปรียบเทียบครบทุกคำสั่ง ก็จะนำค่าของจำนวนคำสั่งที่สอดคล้องกันมาตรวจสอบว่ามากกว่า 1 คำสั่งหรือไม่ ถ้ามากกว่าก็จะไม่เลือกคำสั่งใดทำงาน และรายงานผลกลับไปให้กับผู้ใช้ ถ้าเท่ากับ 1 ก็จะส่งไปให้ฟังก์ชันของแต่ละคำสั่งทำงาน

- ฟังก์ชันที่เหลือจะเป็นฟังก์ชันของแต่ละคำสั่ง โดยบางคำสั่งจะทำงานที่ฟังก์ชันนี้เลย แต่บางคำสั่งจะส่งไปให้คลาส Command จัดการต่อ ซึ่งคำสั่งพวกนี้มักเป็นคำสั่งที่ต้องตรวจสอบอะไรมากมาย

คลาส Command

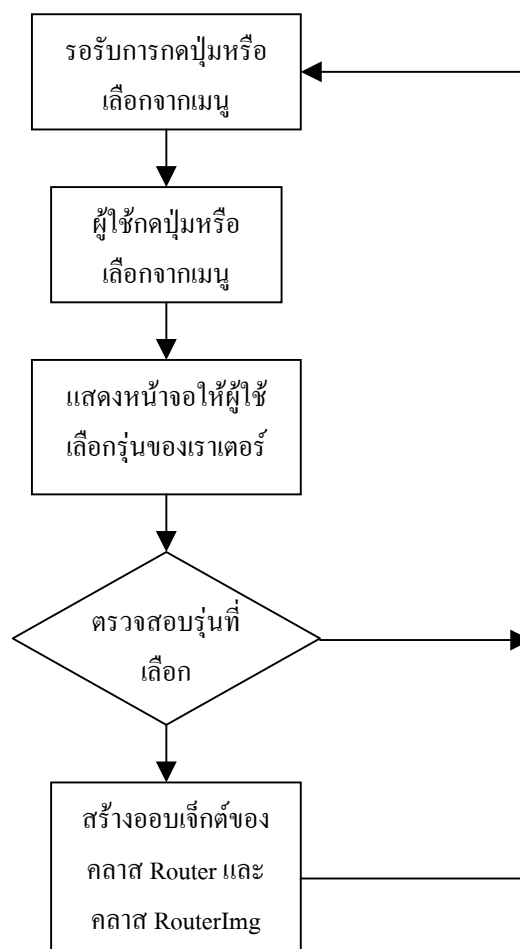
- eraseStartUp() จะเปิดไฟล์ history.dat ขึ้นมาอ่าน แล้วนำคำที่อ่านได้เก็บไว้เรื่อยๆ จนกระทั่งตรวจสอบพบว่าเป็นส่วนที่เก็บค่าของเราเตอร์ตัวที่ต้องการลบค่า ก็จะไม่นำคำที่อ่านมาได้ไปเก็บจนหมดของเราเตอร์ตัวดังกล่าวก็จะอ่านค่าแล้วเก็บไว้เช่นเดิม จนกระทั่งหมดไฟล์ ก็จะปิดไฟล์แล้วเปิดขึ้นมาเพื่อเขียนไฟล์ใหม่ จากนั้นนำคำที่ได้จากการอ่านครั้งที่แล้วเขียนทับลงไปทีไฟล์ history.dat แล้วจึงปิดไฟล์
- ping() ฟังก์ชันนี้จะทำงานแบบรีเคอร์ซีฟ โดยอ่านคำที่เก็บไว้ในตารางค้นหาเส้นทางเพื่อตรวจสอบว่ามีการเก็บข้อมูลสำหรับไปยังจุดหมายปลายทางนั้นๆ แล้วไปตรวจสอบที่เราเตอร์ต่อๆ ไปจนกว่าจะถึงจุดหมายปลายทาง หรือไม่สามารถหาข้อมูลเพื่อไปยังจุดหมายปลายทางนั้นได้
- readStartUp() ฟังก์ชันนี้จะอ่านคำที่เก็บไว้ในไฟล์ history.dat มาแล้วสร้างเราเตอร์ขึ้นมา แล้วกำหนดค่าตามที่กำหนดไว้ในไฟล์ที่เราเตอร์ จนกระทั่งหมดไฟล์
- saveRunningConfig() เช่นเดียวกับฟังก์ชัน eraseStartUp() เมื่อพบเราเตอร์ที่ต้องการจะอ่านค่าที่กำหนดให้กับเราเตอร์ในขณะนั้นมาแทนการอ่านจากไฟล์ แล้วจึงเขียนทับไฟล์เดิม
- show_ip_ospf_inf() ทำหน้าที่แสดงผลจากคำสั่ง show ip ospf <interface name> เพื่อบอกว่าที่อินเทอร์เฟซนั้นๆ มีสถานะของโปรโตคอลโอเอสพีเอฟเป็นเช่นไร
- show_ip_ospf_neighbor() ทำหน้าที่แสดงเราเตอร์ข้างเคียงที่อยู่ในโปรโตคอลโอเอสพีเอฟเช่นเดียวกัน
- showCdp() ฟังก์ชันนี้จะแสดงผลออกมาโดยดึงค่า CDP ที่เก็บไว้ของเราเตอร์ออกมาแสดงผล
- showCdpEntry() ฟังก์ชันนี้จะหาเราเตอร์ข้างเคียงก่อน หลังจากนั้นจึงนำค่า CDP ของเราเตอร์ข้างเคียงมาแสดงผล
- showCdpNeighbors() เช่นเดียวกับ showCdpEntry() แต่ข้อมูลที่นำมาแสดงเป็นข้อมูลคนละประเภทกันคือ เป็นข้อมูลเกี่ยวกับอินเทอร์เฟซที่ต่อกัน
- showHistory() ฟังก์ชันนี้จะนำข้อมูลเกี่ยวกับคำสั่งที่เคยใช้งานของเราเตอร์ออกมาแสดงผล
- showInt() ฟังก์ชันนี้จะแสดงข้อมูลของแต่ละอินเทอร์เฟซของเราเตอร์
- showIPProtocol() ฟังก์ชันนี้จะแสดงข้อมูลโปรโตคอลของเราเตอร์ โดยดูจากค่าตัวแปร RouteType ของเราเตอร์
- showIpRoute() ฟังก์ชันนี้จะแสดงข้อมูลของแต่ละเส้นทางในตารางค้นหาเส้นทาง
- showRunningConfig() ฟังก์ชันนี้จะดูข้อมูลการกำหนดค่าของเราเตอร์ในขณะนั้นมาแสดงผล
- showStartUp() ฟังก์ชันนี้จะทำงานคล้ายกับฟังก์ชัน eraseStartUp() แต่กลับกันโดยเมื่อเจอเราเตอร์ที่ต้องการแล้วจึงจะนำคำที่อ่านได้มาเก็บไว้ แต่หลังจากที่อ่านเสร็จจะไม่มีการเขียนไฟล์ แต่จะนำคำที่อ่านได้แสดงผลออกมาแทน

- showVersion() ฟังก์ชันนี้จะแสดงผลเกี่ยวกับเวอร์ชันของ IOS และข้อมูลของเราเตอร์ โดยอิงจากค่าตัวแปรของเราเตอร์
- telnetCmd() ฟังก์ชันนี้จะตรวจสอบว่าไอพีที่ต้องการเทลเน็ตเข้าไปนั้นถูกต้องหรือไม่ และตรวจสอบต่อว่าพอร์ตเทลเน็ตเปิดไว้หรือไม่
- trace() คล้ายกับฟังก์ชัน ping() แต่จะแสดงผลแต่ละเส้นทางที่ค้นหาได้ออกมาด้วย

4. การจัดการส่วนของการ Add Router

คลาสที่จัดการส่วนนี้คือคลาส Frame1 เป็นการจัดการกับไอคอน โดยโปรแกรมจะดักจับการกดปุ่มเพิ่มเราเตอร์ เมื่อผู้ใช้งานกดปุ่มเพิ่มเราเตอร์ โปรแกรมจะแสดงหน้าต่างให้ผู้เลือกุ่นของเราเตอร์ก่อน ในส่วนนี้ผู้ใช้งานจำเป็นต้องเลือก โปรแกรมจึงจะสร้างออบเจกต์ของเราเตอร์ขึ้นมา พร้อมกับออบเจกต์ของเราเตอร์ที่ใช้ในส่วนของการแสดงภาพเครือข่ายโดยรวมขึ้นมา จากนั้นจะนำเก็บค่าตัวอ้างอิง (Reference) ของพื้นที่นั้นไว้เพื่อใช้สำหรับอ้างอิงภายหลัง

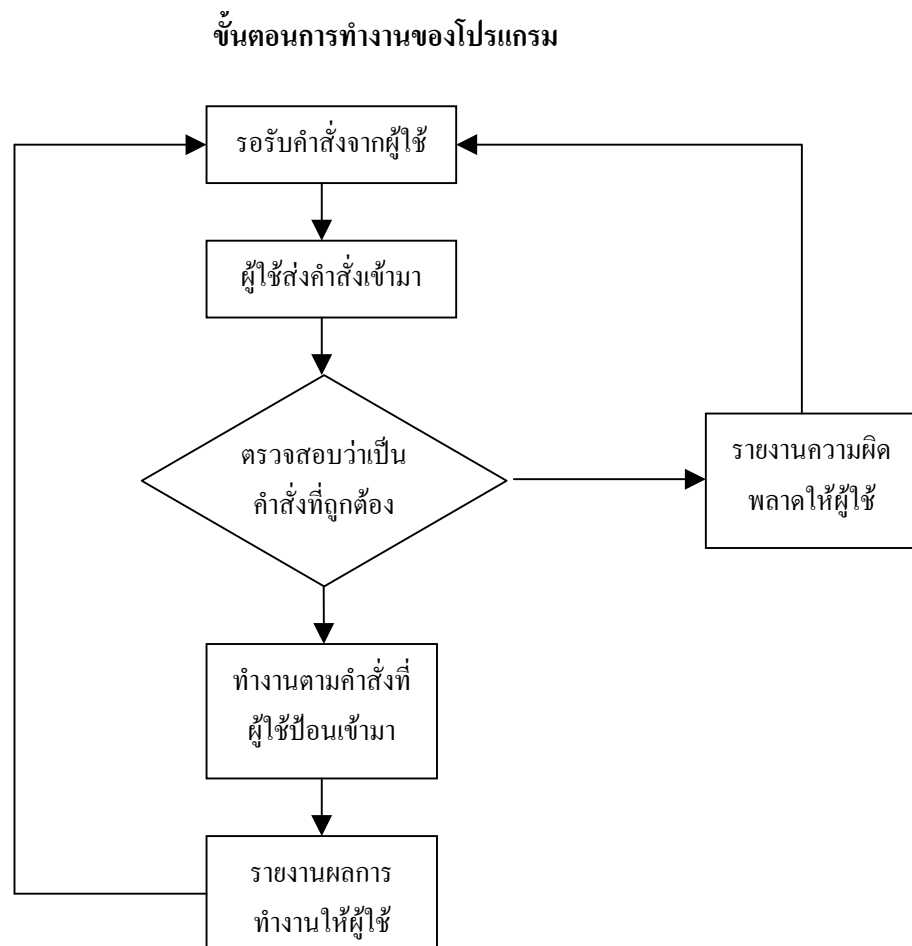
ขั้นตอนการทำงานของโปรแกรม



รูปที่ 6 แสดงขั้นตอนการทำงานของการจัดการเพิ่มเรเตอร์

5. การจัดการส่วนของ Interface

คลาสที่จัดการส่วนนี้คือคลาส Interface โดยการทำงานจะเป็นการทำงานตามคำสั่งที่ผู้ใช้ส่งมา เช่น เพิ่มอินเทอร์เน็ตให้กับเรเตอร์ โปรแกรมจะตรวจสอบค่าไอพีแอดเดรสที่ส่งมากับคำสั่งว่าเป็นค่าที่ใช้
งานได้ แล้วสร้างออบเจกต์ของอินเทอร์เน็ตขึ้นมา แล้วนำค่าอ้างอิงไปเก็บเป็นแบบเวกเตอร์ในออบเจกต์
เรเตอร์ เพื่อใช้ในการอ้างอิงภายหลัง แต่ถ้าเป็นคำสั่งอื่นๆ ที่จัดการกับอินเทอร์เน็ตที่มีอยู่แล้ว โปรแกรม
จะใช้ค่าอ้างอิงออบเจกต์เพื่อหาอินเทอร์เน็ตที่ต้องการ แล้วจึงทำงานกับอินเทอร์เน็ตนั้นๆ ตามคำสั่งที่ส่ง
มา



รูปที่ 7 แสดงขั้นตอนการจัดการอินเทอร์เน็ตของเรเตอร์

ฟังก์ชันการทำงานนี้จะอยู่ในคลาส Interface มีฟังก์ชันการทำงานดังนี้

- Interface() เป็นคอนสตรัคเตอร์ของคลาส Interface ทำหน้าที่กำหนดค่าเริ่มต้นให้กับอินเทอร์เฟซ เช่น ชื่อของอินเทอร์เฟซ เป็นต้น
- AddNewInt() เป็นฟังก์ชันสำหรับเพิ่มอินเทอร์เฟซให้กับเราเตอร์ แต่จะต้องตรวจสอบก่อนว่าจำนวนของอินเทอร์เฟซประเภทนั้นเกินหรือไม่ ถ้าไม่เกินจึงจะสามารถเพิ่มอินเทอร์เฟซได้
- findNetAdd() เป็นฟังก์ชันสำหรับหาค่าเลขประจำเครือข่าย (Net Address) เพื่อนำไปใช้ในการทำงานอื่น สำหรับฟังก์ชันที่ไม่ส่งค่าอาร์กิวเมนต์ใดๆ มาให้ถือว่าให้หาค่าของอินเทอร์เฟซที่เรียกฟังก์ชันนั้น แต่ถ้าส่งค่าไอพีแอดเดรสกับค่าของซับเน็ตมาให้จะหาค่าของเลขประจำเครือข่ายโดยพิจารณาจากค่าทั้ง 2 ค่านี้
- isIp() ฟังก์ชันนี้จะรับอาร์กิวเมนต์มาเป็นสตริง เพื่อนำมาตรวจสอบว่าค่าไอพีแอดเดรสนั้นเป็นค่าไอพีที่ถูกต้องหรือไม่ โดยค่าของไอพีแอดเดรสจะต้องมี 4 ชุด แต่ละชุดจะต้องมีค่าอยู่ระหว่าง 0-255
- noShutInt() เป็นการกำหนดให้อินเทอร์เฟซนั้นอยู่ในสถานะที่ทำงาน
- shutInt() เป็นการกำหนดให้อินเทอร์เฟซอยู่ในสถานะที่ไม่สามารถทำงานได้
- setIpAddress() เป็นการกำหนดค่าไอพีแอดเดรสให้กับอินเทอร์เฟซล่าสุดที่เพิ่งทำงานไป โดยจะดึงอินเทอร์เฟซตัวสุดท้ายมาแล้วกำหนดค่าให้กับอินเทอร์เฟซนั้น

6. การจัดการส่วนของ RIP

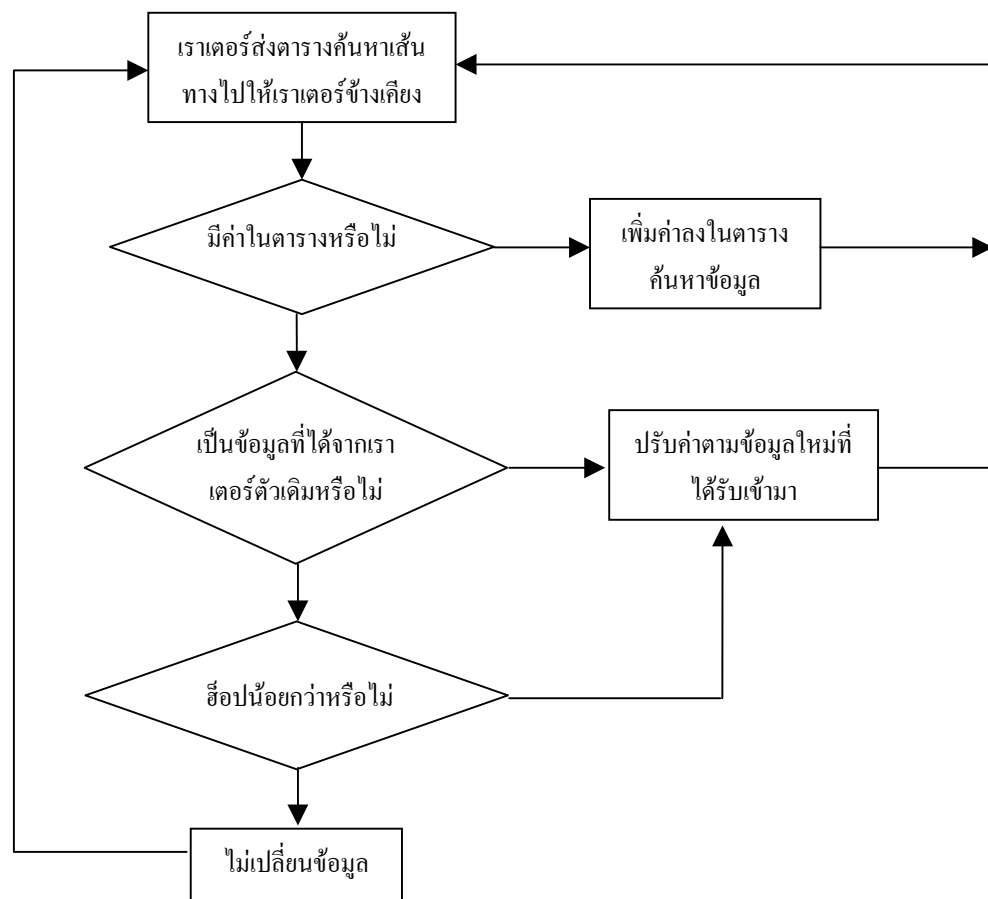
คลาส RIP นี้เป็นคลาสที่สืบทอด (Inherit) มาจากคลาสแม่ที่เป็นเธรด (Thread) เนื่องจากการทำงานโพรโตคอล RIP จะส่งตารางค้นหาเส้นทางให้กันตลอดเวลา ดังนั้นการแตกโปรเซสให้โปรแกรมจัดการกับส่วนที่แลกเปลี่ยนตารางเส้นทางเป็นโปรเซสแบบเธรด จะสิ้นเปลืองเวลาที่ใช้น้อยกว่า รอให้โปรแกรมทำงานเอง สำหรับส่วนของตารางค้นหาเส้นทาง เราเตอร์แต่ละตัวจะมีเวกเตอร์ที่เก็บค่าอ้างอิงของออบเจกต์ของคลาส RoutingTable

วิธีแลกเปลี่ยนตารางค้นหาเส้นทาง เนื่องจากเป็นโปรแกรมจำลองจึงไม่มีการส่งข้อมูลจริงๆ แต่จะตรวจสอบว่าเป็นเราเตอร์ตัวที่อยู่ติดกันเท่านั้นจึงจะสามารถเข้าไปนำข้อมูลของตารางค้นหาเส้นทางออกมาได้ เปรียบเสมือนกับการทำงานจริงๆ ที่เราเตอร์ข้างเคียงเท่านั้นที่จะได้รับข้อมูลตารางค้นหาเส้นทางเมื่อได้ ข้อมูลของตารางค้นหาเส้นทางมาแล้วการพิจารณาว่าจะจัดการกับข้อมูลนั้นอย่างไรจะทำตามโพรโตคอลอาร์ไอพี คือแบ่งการตรวจสอบออกเป็น 3 กรณีดังนี้

1. ถ้าข้อมูลตารางค้นหาเส้นทางที่ได้รับมานั้น เป็นข้อมูลของจุดหมายปลายทางที่ไม่มีอยู่ในตารางค้นหาเส้นทาง โปรแกรมจะเพิ่มเส้นทางนั้นลงในตารางค้นหาเส้นทางโดยสร้างออบเจกต์ของ RoutingTable แล้วเก็บค่าอ้างอิงไว้ที่เราเตอร์
2. ถ้าข้อมูลตารางค้นหาเส้นทางที่ได้รับมานั้น เป็นข้อมูลของจุดหมายปลายทางที่มีอยู่แล้วในตารางค้นหาเส้นทางให้พิจารณาค่าของเมตริกซ์ต่อไปโดย
 - ถ้าค่าเมตริกใหม่มีค่าน้อยกว่าเส้นทางเดิม จะเปลี่ยนค่าในตารางเลือกเส้นทางให้ไปตามทางเส้นทางใหม่

- ถ้าค่าเมตริกใหม่มีค่ามากกว่าหรือเท่ากับเส้นทางเดิม จะไม่เปลี่ยนค่าใดๆ ในตารางค้นหาเส้นทาง
3. ถ้าข้อมูลตารางค้นหาเส้นทางที่ได้รับมานั้น เป็นข้อมูลที่มีเกตเวย์เป็นเราเตอร์ตัวเดียวกันกับเราเตอร์ตัวที่ส่งตารางค้นหาเส้นทางมาให้ ให้ปรับค่าตารางค้นหาเส้นทางนั้นตามค่าที่ได้รับมาใหม่ โดยไม่สนใจว่าค่าเมตริกซ์ใหม่นั้นจะมีค่ามากกว่าหรือน้อยกว่าเส้นทางเดิม

ขั้นตอนการทำงานของโปรแกรม



รูปที่ 8 ขั้นตอนการทำงานของโปรโตคอล RIP

ฟังก์ชันการทำงานนี้จะอยู่ในคลาส RIP และ Router มีฟังก์ชันการทำงานดังนี้

คลาส Rip

- run() เป็นฟังก์ชันสำหรับให้เทรดทำงาน โดยจะทำงานทุกๆ 30 วินาที เพื่อเรียกให้ฟังก์ชันของเราเตอร์ให้ส่งตารางค้นหาเส้นทาง

คลาส Router

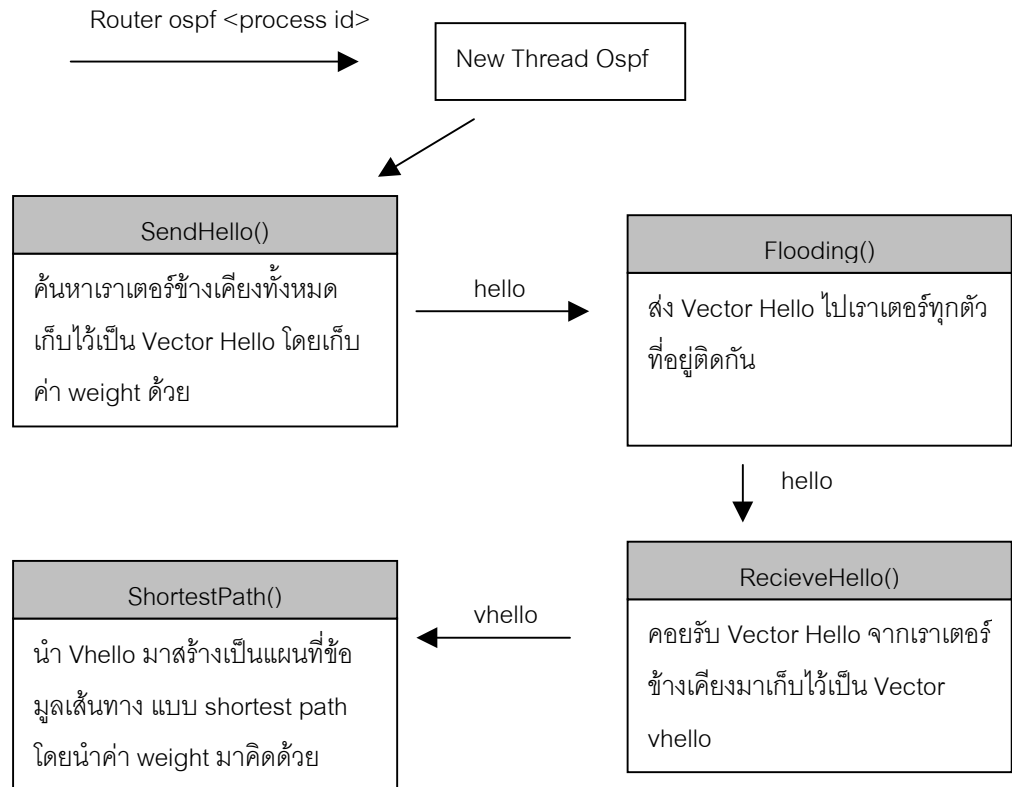
- sendRoutingTable() ฟังก์ชันนี้จะตรวจสอบว่ามีเราเตอร์ที่อยู่ข้างเคียงบ้าง และเราเตอร์นั้นจะต้องใช้โปรโตคอลอาร์ไอพีด้วย แล้วจึงเรียกฟังก์ชัน recieveRoutingTable() เพื่อให้เราเตอร์นั้นรับค่าตารางค้นหาเส้นทาง
- recieveRoutingTable() ฟังก์ชันนี้จะตรวจสอบค่าตารางเส้นทางที่ได้รับมาและปรับเปลี่ยนตามโปรโตคอล

7. การจัดการเกี่ยวกับโปรโตคอลโอเอสพีเอฟ

เมื่อผู้ใช้ป้อนคำสั่ง router ospf <Process id> โปรแกรมจะทำการสร้าง Thread ของคลาส Ospf ขึ้นมาใหม่

การทำงานของโปรโตคอลโอเอสพีเอฟหลังจากสร้าง Thread มีขั้นตอนหลัก 3 ขั้นตอน คือ

1. ตรวจสอบเราเตอร์อื่นที่อยู่ข้างเคียงโดยใช้ฟังก์ชัน sendHello() โดยฟังก์ชันนี้จะทำการตรวจสอบหาเราเตอร์ที่อยู่ข้างเคียงมาเก็บไว้แบบเวกเตอร์
2. เราเตอร์ประกาศเวกเตอร์นี้ออกไปให้เราเตอร์อื่นทุกตัวโดยใช้ฟังก์ชัน Flooding() และเราเตอร์อื่นจะรับโดยใช้ฟังก์ชัน recieveHello() เมื่อเสร็จสิ้นขั้นตอนนี้เราเตอร์ จะทราบว่าเรามีเราเตอร์อื่นใดอยู่ในระบบบ้าง
3. เราเตอร์จะอาศัยข้อมูลจากขั้นที่สองสร้างแผนที่ไปยังเราเตอร์อื่นทุกตัวในเครือข่ายโดยใช้ฟังก์ชัน shortestPath() และเก็บแผนที่ข้อมูลเส้นทางนี้เป็นแบบเวกเตอร์ โดยจะเป็นเส้นทางต้นทางไปยังทุกเราเตอร์ปลายทาง
4. เนื่องจากการทำงานเป็นแบบ Thread จึงทำให้แผนที่ข้อมูลเส้นทางถูกปรับปรุงทุกๆ ช่วงเวลาที่กำหนด



รูปที่ 9 ขั้นตอนการทำงานของโปรโตคอลโอเอสพีเอฟ

ฟังก์ชันการทำงานนี้จะอยู่ในคลาส Ospf และ Router มีฟังก์ชันการทำงานดังนี้

คลาส Ospf

- run() เป็นฟังก์ชันสำหรับให้เธรดทำงาน โดยจะทำงานทุกๆ ช่วงเวลา เพื่อเรียกให้ฟังก์ชันของเราเตอร์ให้ทำฟังก์ชันดังต่อไปนี้

sendHelloPackage(r)	ทำหน้าที่ตรวจสอบหาเราเตอร์ข้างเคียงมาเก็บไว้แบบเวกเตอร์
flooding(r)	ทำหน้าที่ส่งเวกเตอร์ที่ได้จาก sendHelloPackage(r) ไปแบบทุกทิศทางทุกทางที่ติดกับตัวเอง
shortest_path(r)	คำนวณหาระยะทางที่สั้นที่สุดจากตารางข้อมูลที่ได้

คลาส Router

- recieveHelloTable() ทำหน้าที่คอยรับเวกเตอร์ที่ได้จากฟังก์ชัน sendHelloTable() มาเก็บไว้เป็นฐานข้อมูลเพื่อใช้ในการหาเส้นทางที่สั้นที่สุด